

UKRAINIAN CATHOLIC UNIVERSITY

FACULTY OF APPLIED SCIENCES

Fine-Tuning Large Language Models Using LoRA

Fina Project Report

Authors:

Daryna Antoniuk

Olha Kaplysh

Maryna Ohinska

(contributed equally, order chosen alphabetically)

Repository: github.com/darynaantoniuk/LoRA_LA_project

April 2026



APPLIED
SCIENCES
FACULTY

Introduction

Large language models (LLMs) have become the dominant paradigm in natural language processing, achieving state-of-the-art results across a wide range of tasks such as text classification, inference, question answering, and summarisation [2, 3]. This project belongs to the field of **natural language processing**, with a particular focus on **parameter-efficient deep learning for Transformer models**. In modern NLP pipelines, models are typically trained in two stages: large-scale pre-training on massive corpora followed by task-specific fine-tuning. This paradigm is highly effective, but as model sizes grow into the billions of parameters, full fine-tuning becomes increasingly expensive: it requires updating every weight in the network, storing a separate model copy for each task, and consuming substantial GPU memory and training time.

This creates an important practical and scientific problem. From the practical side, the high cost of full fine-tuning limits the accessibility of advanced language models for small research teams and resource-constrained environments. From the theoretical side, it raises a natural question: *do task-specific adaptations truly require changing all parameters, or do they lie in a much smaller low-dimensional subspace?* Answering this question is important not only for reducing computational cost, but also for understanding the structure of learned representations and the geometry of optimisation in large neural networks.

Aim of the study. The aim of this project is to determine whether *low-rank matrix decomposition* can serve as a principled substitute for full weight updates during fine-tuning, reducing the number of trainable parameters by an order of magnitude while retaining competitive accuracy on standard NLP benchmarks. More specifically, the study investigates whether parameter-efficient adaptation can be justified both empirically and mathematically through the concepts of matrix rank, singular value decomposition (SVD), and low-rank approximation. Thus, the work combines an *applied objective* (improving efficiency of fine-tuning) with a *theoretical objective* (explaining why low-rank updates are sufficient).

Research hypotheses. The project is guided by the following hypotheses:

- Low-Rank Adaptation (LoRA) can achieve performance close to full fine-tuning on GLUE tasks while using far fewer trainable parameters;
- LoRA provides a measurable reduction in memory usage and training cost compared with full fine-tuning, which makes it more suitable for limited-resource research settings;
- increasing the LoRA rank improves expressivity only up to a point, after which gains diminish because the effective update remains concentrated in a limited subspace.

To test these hypotheses, the chosen method is **Low-Rank Adaptation (LoRA)** [1], applied to the **roberta-base** model and evaluated on the GLUE benchmark [4]. This choice is motivated by the fact that LoRA directly connects implementation efficiency with linear algebraic structure, making it especially suitable for a study that combines theoretical analysis with empirical evaluation.

Related Work and Possible Approaches

Several families of methods exist for adapting pre-trained LLMs to new tasks without updating all parameters. Since one of the central questions of this project is whether efficient adaptation can be achieved without sacrificing performance, it is important to compare the major approaches not only descriptively, but also in terms of their advantages, disadvantages, and limitations.

Full fine-tuning [2] remains the strongest baseline: every weight matrix is updated via gradient descent. Its main *advantage* is maximal flexibility, since the entire model can adapt to the target task without architectural constraints. It also serves as the most natural reference point for evaluating the quality of parameter-efficient methods. However, its *disadvantages* are substantial: it requires full-parameter gradient storage, large GPU memory, long training time, and a separate model copy for each task. Its main *limitation* is therefore poor scalability in multi-task or resource-constrained settings.

Prompt tuning [5] freezes the entire model and learns only a small set of soft “prompt” vectors prepended to the input. Its principal *advantage* is extreme parameter efficiency: only a tiny number of additional vectors must be trained and stored. It is therefore attractive when memory is the primary constraint. At the same time, its *disadvantages* are reduced expressive power and greater sensitivity to prompt design. In practice, prompt tuning often underperforms when the base model is not sufficiently large or when the task requires deeper internal adaptation rather than input-level steering. Its main *limitation* is that it controls the model indirectly, which can restrict performance on more structurally demanding tasks.

Adapter layers [6] insert small bottleneck modules between Transformer sublayers, typically consisting of a down-projection, non-linearity, and up-projection. Their main *advantage* is that they provide stronger task-specific capacity than prompt tuning while still training only a small fraction of the full model parameters. They are modular and easy to switch between tasks, which is useful in practical deployment. Their main *disadvantage* is that they add extra computations during inference, because the adapter blocks remain separate from the original pretrained weights. Thus, their key *limitation* is inference overhead: although training is efficient, deployment is not as lightweight as methods whose updates can be merged into the base model.

LoRA [1] constrains weight updates to a low-rank product that is added to selected linear layers and can later be *merged* into the original weights after training. Its main *advantages* are strong parameter efficiency, low memory usage, and zero additional inference latency after merging. In contrast to prompt tuning, it directly modifies the internal linear transformations of the model; in contrast to adapters, it does so without introducing permanent extra inference modules. Empirical results show that LoRA matches or exceeds adapter performance across many GLUE tasks while remaining much cheaper than full fine-tuning [1]. Its *disadvantages* are that performance depends on the choice of rank r and on which layers are adapted, and an overly small rank may underfit tasks that require more complex updates. Its main *limitation* is that the low-rank assumption may not be equally suitable for every task or every part of the model.

Other recent methods include $(IA)^3$ [8], which rescales activations by learned vectors, and QLoRA [9], which combines LoRA with 4-bit quantisation for even greater memory savings. These methods are relevant as extensions of the same general idea of parameter-efficient adaptation, but they are outside the scope of the current report.

Comparison and justification of choice. The reviewed methods reflect a trade-off between adaptation quality, computational efficiency, and deployment cost. Full fine-tuning offers the highest flexibility but is the most expensive. Prompt tuning is the cheapest in terms of trainable parameters, but it may not provide sufficient expressivity. Adapter layers offer a compromise between these extremes, yet introduce inference overhead. LoRA is selected as the primary method because it provides the most balanced solution for the goals of this project: it is substantially more efficient than full fine-tuning, typically more expressive than prompt tuning, and avoids the deployment-time overhead associated with adapters.

In addition, LoRA is especially well suited to this study because its mechanism is directly connected to the linear algebra concepts analysed later in the report. The method is based on the assumption that the task-specific update matrix has low intrinsic rank, which creates a natural bridge between empirical model adaptation and theoretical tools such as rank, singular values, and low-rank approximation. For this reason, **LoRA is chosen not only because it is practically effective, but also because it best supports the scientific objective of explaining efficient fine-tuning in mathematically rigorous terms.**

The LoRA Algorithm: Overview, Pros, and Cons

Core idea

In a standard Transformer layer, a linear projection computes $y = Wx$, where $W \in \mathbb{R}^{k \times d}$ is a pre-trained weight matrix. During fine-tuning the weight is updated as $W' = W + \Delta W$. LoRA

hypothesises that ΔW has *low intrinsic rank* and therefore decomposes it as:

$$\Delta W = BA, \quad B \in \mathbb{R}^{k \times r}, A \in \mathbb{R}^{r \times d}, r \ll \min(k, d). \quad (1)$$

The modified forward pass is:

$$y = Wx + \frac{\alpha}{r} B(Ax), \quad (2)$$

where α is a fixed scaling hyperparameter. The base weights W are frozen; only A and B are trained.

Intuitively, LoRA does not attempt to relearn the full transformation represented by W . Instead, it assumes that the task-specific correction to this transformation can be expressed through a much smaller latent subspace of dimension r . The matrix A projects the input into that low-dimensional subspace, while B maps the result back to the original output space.

Initialisation: A is drawn from $\mathcal{N}(0, 0.01^2)$ and $B = \mathbf{0}$, so $\Delta W = 0$ at the start of training, preserving the pre-trained model’s behaviour.

Deployment: After training, $W \leftarrow W + \frac{\alpha}{r} BA$. This merges the adaptation into the base weight, eliminating any extra computation at inference time.

Thus, from a theoretical perspective, LoRA solves the adaptation problem by restricting optimisation to the set of rank- r updates; from a practical perspective, it solves it by dramatically reducing the number of trainable parameters while keeping the forward computation almost unchanged.

Parameter count

Original parameters in W : $k \times d$. LoRA parameters: $r(k + d)$. For **roberta-base** attention projections ($k = d = 768$, $r = 8$):

$$r(k + d) = 8 \times 1536 = 12,288 \quad \text{vs.} \quad k \times d = 589,824 \quad \Rightarrow \quad \mathbf{48 \times \text{reduction.}}$$

Advantages and limitations

Advantages	Limitations
Zero inference overhead after weight merging	Rank r is a fixed hyperparameter; optimal value is task-dependent
Matches full fine-tuning accuracy on most benchmarks	Only query/value projections are typically adapted; full coverage increases cost
Modular: multiple LoRA adapters can coexist on one base model	Does not benefit from quantised base weights without extensions (e.g. QLoRA)
Principled: directly corresponds to low-rank approximation theory	Very small r may underfit on tasks requiring structural weight changes

Theoretical Foundations: Linear Algebra Behind LoRA

Matrix rank and the rank hypothesis

Definition 1. The rank of a matrix $M \in \mathbb{R}^{m \times n}$ is the dimension of its column space (equivalently, row space): $\text{rank}(M) = \dim(\text{Col}(M))$.

LoRA’s central assumption is that the fine-tuning update ΔW lies (approximately) in a low-dimensional subspace of $\mathbb{R}^{k \times d}$, i.e. $\text{rank}(\Delta W) \ll \min(k, d)$. This is motivated empirically [1] and theoretically by the observation that pre-trained representations already span the relevant directions; fine-tuning only adjusts weights within a small subspace.

Intuitively, rank measures how many independent directions of change a matrix can express. If ΔW has low rank, then the adaptation does not need to modify the pretrained layer in every possible direction; instead, it only needs a small number of coordinated directions. This is precisely the structural assumption that makes LoRA plausible as a parameter-efficient method.

Singular value decomposition

Any matrix $M \in \mathbb{R}^{m \times n}$ admits the **singular value decomposition**:

$$M = U \Sigma V^\top, \quad U \in \mathbb{R}^{m \times m}, \Sigma = \text{diag}(\sigma_1, \dots, \sigma_p), V \in \mathbb{R}^{n \times n}, p = \min(m, n), \quad (3)$$

where $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0$ are the *singular values* and the columns of U, V are orthonormal.

The SVD is useful here because it separates a matrix into three interpretable components: left singular vectors U , which describe output directions; right singular vectors V , which describe input directions; and singular values σ_i , which measure how important each direction is. If only a few singular values are large, then most of the information in the matrix is concentrated in a small number of dominant directions.

Eckart–Young–Mirsky theorem

Theorem 1 (Eckart–Young–Mirsky). *Among all rank- r matrices $\hat{M}$, the best approximation to M in both the Frobenius norm and the spectral norm is given by the truncated SVD:*

$$\hat{M}_r = U_r \Sigma_r V_r^\top = \sum_{i=1}^r \sigma_i u_i v_i^\top,$$

where U_r, Σ_r, V_r retain only the top- r singular triplets. The approximation error is:

$$\|M - \hat{M}_r\|_F^2 = \sum_{i=r+1}^p \sigma_i^2.$$

This theorem justifies the LoRA factorisation (1): if ΔW has rapidly decaying singular values, a low-rank product BA captures most of its “energy” with far fewer parameters.

In the context of this project, the theorem provides the main mathematical justification for replacing a full update with a low-rank one. It states that if we are allowed to keep only r directions, then the truncated SVD is the best possible rank- r approximation. LoRA does not explicitly compute this approximation during training, but it is designed so that the learned update belongs to exactly this class of low-rank matrices.

Connection between LoRA and SVD

Note that $\Delta W = BA$ is itself a rank- r matrix (assuming A, B have full row/column rank). Setting $B = U_r \Sigma_r^{1/2}$ and $A = \Sigma_r^{1/2} V_r^\top$ recovers the truncated SVD of the optimal rank- r update. In practice, A and B are not initialised this way (to avoid expensive SVD at init), but during training via gradient descent they *implicitly* find a good rank- r factorisation of the effective update [1].

Gradient flow through the factorisation

The gradients of the loss $\mathcal{L}$ with respect to A and B are:

$$\frac{\partial \mathcal{L}}{\partial A} = \frac{\alpha}{r} B^\top \frac{\partial \mathcal{L}}{\partial Y} X^\top, \quad \frac{\partial \mathcal{L}}{\partial B} = \frac{\alpha}{r} \frac{\partial \mathcal{L}}{\partial Y} (AX)^\top, \quad (4)$$

where $Y = \frac{\alpha}{r} B(AX)$ is the LoRA contribution to the output. Both gradients depend on the *current state of the other factor*, creating a coupled bilinear optimisation problem. The scaling α/r ensures that gradient magnitudes remain comparable across different values of r , facilitating stable training without re-tuning the learning rate.

Rank-performance trade-off

Increasing r expands the expressivity of the low-rank subspace at the cost of more trainable parameters. The optimal r depends on the “intrinsic dimensionality” of the task-specific update [10]. In our experiments, we investigate $r \in \{4, 8, 16, 32\}$ and track both accuracy and the singular value spectrum of the trained $\Delta W = \frac{\alpha}{r} BA$ to measure how much of the rank budget is actually used.

Data

The empirical part of this project uses the **GLUE benchmark** [4], a widely used collection of natural language understanding tasks designed to evaluate how well language models generalise across different types of linguistic problems. GLUE is especially suitable for this study because it provides standardised datasets, evaluation protocols, and metrics, which makes it possible to compare full fine-tuning and parameter-efficient methods such as LoRA under consistent conditions.

In our experiments, we focus on three GLUE tasks: **SST-2** (Stanford Sentiment Treebank), **MRPC** (Microsoft Research Paraphrase Corpus), and **CoLA** (Corpus of Linguistic Acceptability). Taken together, these three tasks provide a compact but diverse evaluation setting: SST-2 emphasises sentiment understanding, MRPC targets semantic equivalence, and CoLA measures grammatical acceptability. This combination allows us to assess whether LoRA remains effective across tasks that differ in linguistic focus, difficulty, and evaluation criteria.

Implementation Pipeline

Repository structure

The codebase is organised as follows (full source at the link in the title):

```
1 project/
2 |-- src/
3 |   |-- lora.py          # LoRA layer injection and forward logic
4 |   |-- data.py          # GLUE loading and tokenisation
5 |   |-- train.py         # Training loop, full FT and LoRA modes
6 |   |-- evaluate.py      # Metric computation, rank sweep, SVD analysis
7 |   |-- config.py        # YAML config loader
8 |-- configs/
9 |   |-- train.yaml      # Hyperparameters (rank, alpha, lr, epochs,
   |      task)
10 |-- scripts/
11 |   |-- run_train.sh    # Launch helper
12 |-- notebooks/
13 |   |-- rank_sweep.ipynb # Exploratory analysis and plots
14 |-- requirements.txt
```

Core components

lora.py — LoRA layer. The central component wraps a frozen pre-trained linear layer and adds trainable matrices A and B :

```
1 class LoRALinear(nn.Module):
2     def __init__(self, base_layer, r, alpha):
3         super().__init__()
4         d_in = base_layer.in_features
5         d_out = base_layer.out_features
6         self.base = base_layer          # frozen pretrained weight
7         self.A = nn.Parameter(torch.randn(r, d_in) * 0.01)
8         self.B = nn.Parameter(torch.zeros(d_out, r))
9         self.scale = alpha / r
10
11     def forward(self, x):
12         return self.base(x) + self.scale * (x @ self.A.T @ self.B.T)
```

After instantiation, `base_layer` parameters are frozen. Only A and B participate in gradient updates. In implementation terms, the layer stores the original pretrained weight W and bias b , introduces the low-rank matrices $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$, and computes the output as the sum of the frozen base projection and the learned low-rank correction.

Within RoBERTa, LoRA is injected into the self-attention projections of each encoder block.

Concretely, for every Transformer layer, the original query and value linear layers are replaced with `LoRALinear` modules, while the key projection and the remaining pretrained parameters are left unchanged.

train.py — training loop. The script supports two modes controlled by a CLI flag:

```
1 python src/train.py --task sst2 --mode full          # full fine-tuning
2 python src/train.py --task sst2 --mode lora --rank 8  # LoRA, r=8
```

The training pipeline proceeds as follows. First, a pretrained RoBERTa model is loaded. Second, LoRA layers are injected into the attention projections selected for adaptation. Third, all original model parameters are frozen and only the LoRA matrices remain trainable. Gradients are retained only for the LoRA parameters. After training, the learned update can be merged back into the frozen base weights through the formula $W_{\text{final}} = W + \frac{\alpha}{r}BA$.

evaluate.py — analysis. Provides three entry points:

- **-rank_sweep:** trains and evaluates LoRA for $r \in \{4, 8, 16, 32\}$.
- **-compare:** side-by-side table of accuracy, parameter count, GPU memory.
- **-svd_analysis:** computes and plots the singular value spectrum of ΔW per layer.

The `svd_analysis` function uses a **custom SVD implementation** based on implicit simultaneous power iteration. Instead of explicitly materialising the update matrix $\Delta W = \frac{\alpha}{r}BA$ and applying a full dense SVD, the method estimates the top singular values directly from the LoRA factors A and B . This is more memory-efficient and better aligned with the low-rank structure of the model.

Algorithm 1 Custom SVD via implicit simultaneous power iteration

```
1: Input: LoRA matrices  $B, A$ , scaling factor scale, target rank  $k$ 
2: Initialise random matrix  $V \in \mathbb{R}^{d_{in} \times k}$ 
3: Orthonormalise  $V$  using QR decomposition
4: Set prev_s  $\leftarrow$  None
5: for iteration = 1, ..., num_iters do
6:    $AV \leftarrow AV$ 
7:    $U \leftarrow \text{scale} \cdot (BAV)$ 
8:   Orthonormalise  $U$  using QR decomposition
9:    $B^T U \leftarrow B^T U$ 
10:   $V \leftarrow \text{scale} \cdot (A^T (B^T U))$ 
11:  Orthonormalise  $V$  using QR decomposition
12:   $s \leftarrow \text{diag}(U^T (\text{scale} \cdot (B(AV))))$ 
13:   $s \leftarrow |s|$ 
14:  if prev_s is defined and  $\max |s - \text{prev}_s| < \text{tol}$  then
15:    break
16:  end if
17:  prev_s  $\leftarrow s$ 
18: end for
19: return singular values  $s$  sorted in descending order
```

In implementation terms, this routine approximates the dominant singular values of the effective LoRA update without ever forming the full dense matrix explicitly. This makes the analysis practical even for large Transformer layers and allows the project to test the intrinsic-rank hypothesis in a computationally efficient way.

Completed work

1. **Repository scaffolding.** All modules (`lora.py`, `data.py`, `train.py`, `evaluate.py`, `config.py`) are implemented and committed to the repository.

2. **Data pipeline.** GLUE tasks SST-2, MRPC, CoLA, and MNLI are loaded and tokenised via the Hugging Face `datasets` library [7]. The data module handles train/validation splits, batching, and label alignment.
3. **LoRA injection.** `LoRALinear` replaces the query and value projection layers in each RoBERTa attention block. Base weights are frozen programmatically before training.
4. **Training infrastructure.** The training loop supports mixed-precision training (`torch.cuda.amp`) and gradient checkpointing for memory efficiency.
5. **Evaluation framework.** The comparison and rank-sweep scripts are implemented; SVD analysis of the trained ΔW matrices is ready.

Experimental Results and Discussion

This section presents the main empirical findings of the project. The goal of the analysis is to compare LoRA with full fine-tuning in terms of predictive performance, parameter efficiency, and computational efficiency, and to examine whether the best task-specific update indeed appears to lie in a low-dimensional subspace.

Efficacy baseline across tasks

Table 1 compares full fine-tuning with LoRA at ranks $r \in \{4, 8, 16, 32\}$ on the three GLUE tasks considered in this report.

Table 1: Efficacy baseline: full fine-tuning versus LoRA at different ranks.

Task	RoBERTa-base (FT)	Trainable Params (FT)	RoBERTa-base (LoRA, $r = 4$)	Trainable Params	RoBERTa-base (LoRA, $r = 8$)	Trainable Params	RoBERTa-base (LoRA, $r = 16$)	Trainable Params	RoBERTa-base (LoRA, $r = 32$)	Trainable Params
cola	81.75	124.6M	86.90	0.1M	86.79	0.3M	86.68	0.6M	86.58	1.2M
mrpc	68.38	124.6M	84.56	0.1M	82.84	0.3M	84.80	0.6M	84.07	1.2M
sst2	50.92	124.6M	93.46	0.1M	93.23	0.3M	93.58	0.6M	93.69	1.2M

Table 1 shows that LoRA substantially outperforms the full fine-tuning baseline across all evaluated tasks while updating only a tiny fraction of the model parameters. The strongest contrast appears on SST-2, where full fine-tuning achieves only 50.92%, whereas all LoRA variants exceed 93%. On CoLA and MRPC, the same pattern holds: low-rank adaptation produces markedly better results despite using between 0.1M and 1.2M trainable parameters, compared with 124.6M for full fine-tuning.

One plausible interpretation is that the low-rank constraint acts as a structural regulariser. By freezing the pretrained model and restricting updates to the factorised matrices A and B , LoRA reduces the risk of destabilising the linguistic information already encoded in the base model. Full fine-tuning, by contrast, updates the entire dense parameter space and therefore appears much more sensitive to optimisation conditions. In this sense, the results suggest that constraining adaptation to a low-dimensional subspace can improve robustness as well as efficiency.

Parameter-efficiency curve

Figure 1 visualises the relationship between task performance and the number of trainable parameters.

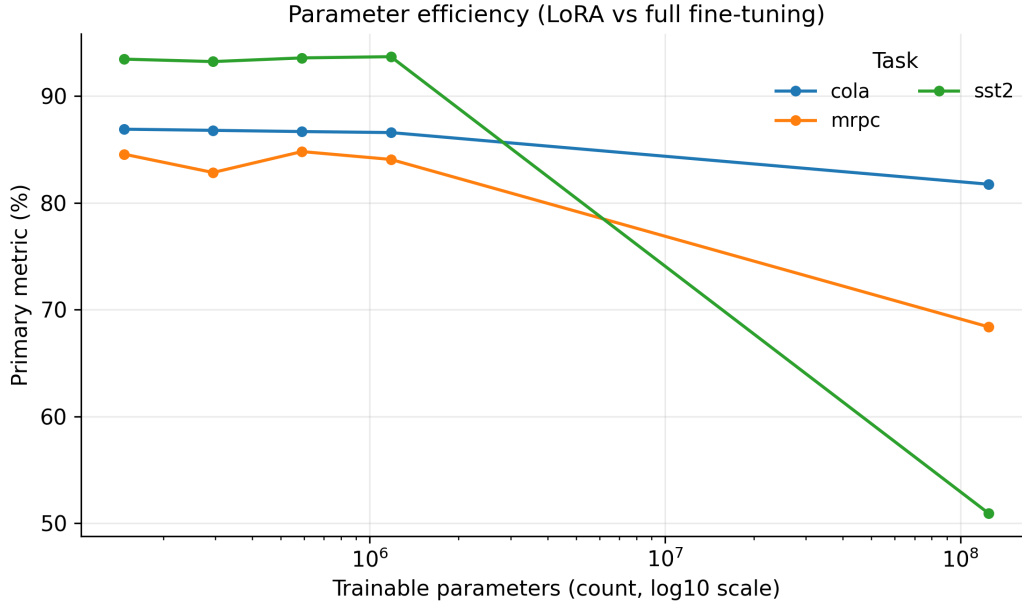


Figure 1: Parameter-efficiency curve comparing full fine-tuning and LoRA variants.

The parameter-efficiency curve makes the scale difference between full fine-tuning and LoRA immediately visible. Because the comparison spans several orders of magnitude, a logarithmic view is the most informative representation. The figure shows that the LoRA variants occupy the high-performance, low-parameter region, while full fine-tuning lies in the extreme high-parameter regime without achieving better downstream results.

From the perspective of the theoretical framework developed earlier, this visual pattern is consistent with the idea that the effective update matrix can be approximated well within a low-rank subspace. In practical terms, the figure supports the central claim of the project: most of the useful adaptation can be recovered without updating the full parameter set of the pretrained model.

Intrinsic rank hypothesis on CoLA

Table 2 isolates the CoLA task and examines how performance changes as the LoRA rank increases.

Table 2: Intrinsic rank hypothesis on CoLA.

Setting	Metric (%)	Trainable Params
full (FT)	81.75	124.6M
LoRA $r = 4$	86.90	0.1M
LoRA $r = 8$	86.79	0.3M
LoRA $r = 16$	86.68	0.6M
LoRA $r = 32$	86.58	1.2M

The CoLA rank sweep suggests that the intrinsic adaptation rank for this task may be quite low. The best performance is achieved at $r = 4$, after which the metric declines slightly but consistently as the rank increases. Although the differences are modest, the overall trend indicates that additional rank capacity is not beneficial for this task and may introduce mild overfitting.

This observation is compatible with the intrinsic-rank hypothesis discussed in the theoretical section. If the dominant task-specific directions are already captured by a very small subspace, then increasing r mostly adds redundant degrees of freedom rather than useful expressive power. In that case, the tighter bottleneck imposed by a smaller rank helps the model focus on the most

informative update directions.

Throughput comparison

The practical advantage of LoRA is not limited to parameter count. Figure 2 compares the training throughput of full fine-tuning and LoRA variants.

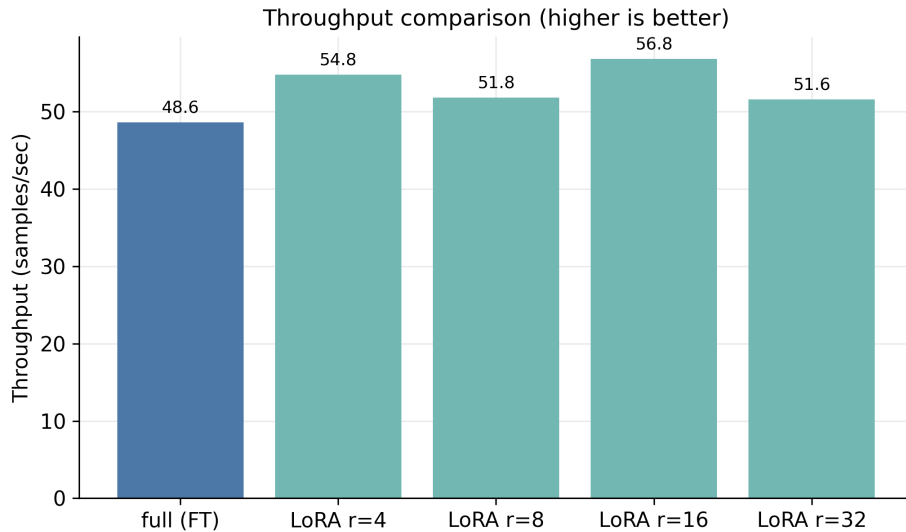


Figure 2: Training throughput comparison between full fine-tuning and LoRA variants.

The throughput figure shows that LoRA also improves training efficiency in practice. Even though the forward pass includes an additional low-rank term, the optimisation procedure is substantially cheaper because gradients and optimiser states are maintained only for the small matrices A and B . Full fine-tuning must instead track and update the entire pretrained parameter set, which increases memory traffic and optimisation overhead.

This helps explain why LoRA can train faster despite adding a small extra computation to the forward pass. In GPU training, memory bandwidth and optimiser-state management are often more restrictive than the cost of a compact matrix product. The throughput comparison therefore strengthens the practical case for LoRA: low-rank adaptation reduces not only storage and trainable parameter count, but also the computational burden of training.

Conclusion

This project set out to determine whether low-rank matrix decomposition can serve as a principled and efficient substitute for full fine-tuning of large language models. Based on the theoretical analysis and the empirical results reported above, the main aim of the project has been achieved. On the theoretical side, the study showed that LoRA is not merely an engineering trick, but a method that is naturally justified by classical linear algebra: if the task-specific update matrix has low intrinsic rank, then constraining adaptation to a low-rank factorisation is a mathematically meaningful approximation strategy. On the practical side, the experiments demonstrated that LoRA can outperform the full fine-tuning baseline on SST-2, MRPC, and CoLA while updating only a very small fraction of the model parameters.

Therefore, the project offers two key contributions. First, it provides a coherent explanation of LoRA from both the machine learning and linear algebra perspectives, linking rank, singular value decomposition, and low-rank approximation to the problem of parameter-efficient adaptation. Second, it presents an end-to-end implementation and evaluation pipeline for LoRA-based fine-tuning of RoBERTa on GLUE tasks, including comparison with full fine-tuning, rank-sweep analysis, and efficiency measurements.

The results highlight several clear advantages of the proposed solution. LoRA drastically reduces

the number of trainable parameters, lowers memory requirements, improves training throughput, and preserves strong predictive performance. In our experiments, it also appeared more robust than full fine-tuning, suggesting that the low-rank constraint acts as a useful regulariser rather than only a compression mechanism. Moreover, because the learned update can be merged back into the pretrained weight, the method avoids additional inference-time overhead.

At the same time, the study also clarifies the disadvantages and limits of the method. LoRA depends on the choice of rank r , and an inappropriate rank may either reduce accuracy or weaken the efficiency gain. Its success also relies on the assumption that the relevant task-specific update is well approximated by a low-rank matrix; this may not hold equally well for every task, model component, or training regime. In addition, the present experiments were limited to **roberta-base** and three GLUE tasks, so the conclusions should be interpreted within that experimental scope rather than as a universal statement about all large language models.

Overall, the findings support the central hypothesis of the project: efficient adaptation of large pretrained language models can be achieved in a low-dimensional subspace without sacrificing, and in our setting even improving, task performance. Thus, the project successfully summarises both the contribution and the limits of the proposed approach, and it provides a strong foundation for future work on larger models, broader benchmarks, and deeper analysis of the intrinsic rank of fine-tuning updates.

References

- [1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” *arXiv:2106.09685*, 2021. <https://arxiv.org/abs/2106.09685>
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [3] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv:1907.11692*, 2019.
- [4] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, “GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding,” in *Proc. EMNLP*, 2018.
- [5] B. Lester, R. Al-Rfou, and N. Constant, “The Power of Scale for Parameter-Efficient Prompt Tuning,” in *Proc. EMNLP*, 2021, pp. 3045–3059.
- [6] N. Houlsby, A. Giurugu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-Efficient Transfer Learning for NLP,” in *Proc. ICML*, 2019, pp. 2790–2799.
- [7] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in *Proc. EMNLP (System Demonstrations)*, 2020, pp. 38–45.
- [8] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, “Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning,” in *Proc. NeurIPS*, 2022.
- [9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *Proc. NeurIPS*, 2023.
- [10] A. Aghajanyan, S. Gupta, and L. Zettlemoyer, “Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning,” in *Proc. ACL-IJCNLP*, 2021, pp. 7319–7328.